Дипломная работа

на тему:

“Анализ и сравнение написания web-приложений с использованием разных фреймворков”

Выполнил: Яковлев Руслан Евгеньевич

Содержание

1. Введение.....3
2. Введение в фреймворки.....4
   - Что такое Flask?.....4
   - Что такое FastAPI?.....6
   - Что такое Django?.....7
3. Пример работы программы Django, FastAPI и Flask.....9
   - Пример работы на Flask.....9
   - Пример работы на FastAPI.....10
   - Пример работы на Django.....11
   - Шаблоны HTML.....14
4. Сравнение работы на Flask, Django и FastAPI.....15
   - Сравнение Django и Flask.....15
   - Сравнение Django и FastAPI.....16
   - Сравнение FastAPI и Flask.....18
5. Приложение 1. Пример файловой структуры.....21
6. Приложение 2. Список необходимых библиотек.....22

Введение.

Почему я выбрал тему **“Анализ и сравнение написания web-приложений с использованием разных фреймворков”?**

В современном мире технологии развиваются невероятно быстро, и новые языки программирования не заставляют себя долго ждать. За последние несколько лет, появилось больше 700 языков программирования, и продолжают появляться по сей день, но не все из них являются достаточно удобными для пользования. И вот появились три относительно новых инструмента, которые помогают дополнять и разнообразить HTML шаблоны, этими инструментами являются Django(2005 г.), FastAPI(2018 г.) и Flask(2010 г.).

HTML шаблоны — это важная часть любого сайта. Без этих шаблонов мир никогда бы не увидел самые разнообразные сайты, и некоторые приложения, работающие на языке HTML или на другом языке программирования.

Благодаря множеству разных фреймворков эти самые HTML шаблоны стали более гибкими в использовании, и появилось гораздо больше новых возможностей, если сравнивать изначально.

Фреймворк — это набор готовых инструментов, библиотек и функций, которые помогают разработчикам создавать различные веб-сайты и приложения. Фреймворк предоставляет готовую структуру, на основе которой можно быстро и легко создать сайт или какую-нибудь другую программу. В нынешнее время количество фреймворков перевалило за пятьдесят, и все они пользуются определённой популярностью. Этими фреймворками как раз являются инструменты, упомянутые выше, то есть Django(2005 г.), FastAPI(2018 г.) и Flask(2010 г.).

Но что из всего этого больше подойдёт лично вам? Какой из них самый удобный? Какой самый практичный? Этот проект как раз должен пролить свет на все эти вопросы. В этом проекте будет сравнение этих трёх фреймворков между собой и я постараюсь ответить на все ранее заданные вопросы.

Введение в фреймворки

Что бы лучше понять о чём пойдёт речь в этой дипломной работе нужно понимать о том, как работает Flask, Django и FastAPI. Так что для полноты картины, были описаны ниже принципы работы, а также различные расширения в каждом фреймворке, для того чтобы лучше понимать то, о чём в этом проекте идёт речь.

Что такое Flask?

Flask — это фреймворк для разработки веб-приложений на языке программирования Python. Он предоставляет собой базовые функции и инструменты для создания сайтов и приложений, но при этом оставляет разработчику большую свободу действий и гибкость в настройке проекта.

Фреймворк Flask не имеет большого количества встроенных функций, как у других более крупных фреймворков, но он позволяет легко расширять функциональность приложения с помощью сторонних библиотек и модулей. Основные особенности Flask это лёгкость и простота. Flask имеет небольшой размер и простую структуру, что делает его лёгким в изучении и использовании на практике.

Плюсы фреймворка Flask:

1. **Гибкость**. Разработчик может настроить Flask под свои нужды, выбрав нужные модули и библиотеки.
2. **Расширяемость**. Flask совместим со многими сторонними библиотеками и инструментами, что позволяет расширить функциональность приложения.
3. **Сообщество**. Flask имеет активное сообщество разработчиков, которые делятся опытом, советами и решениями проблем.

Для работы с этим фреймворком необходимо как минимум знать язык программирования Python и основы веб-разработки на HTML чтобы создавать свои собственные проекты на Flask. Вот несколько примеров того, какие проекты можно создать во Flask, Django и FastAPI:

1. Создание простого блога или новостного сайта
2. Разработка интернет-магазина с небольшим ассортиментом товаров
3. Создание системы управления контентом (CMS) для небольшого сайта
4. Разработка API для взаимодействия с другими приложениями
5. Создание приложения для обработки данных и предоставления результатов в виде веб-интерфейса
6. Разработка платформы для проведения опросов или голосований
7. Создание сервиса для обмена файлами между пользователями
8. Разработка системы учёта рабочего времени сотрудников офисных кампаний и не только
9. Создание платформы для онлайн-обучения
10. Разработка приложения для управления проектами

И здесь далеко не все примеры использования этих библиотек. Возможности этого фреймворка ограничены только воображением самого разработчика ну и ещё RESTfull API, но об этом позже.

Flask состоит из нескольких основных компонентов:

- **Маршрутизация**. Это механизм, который определяет, какой код будет выполняться при получении определённого запроса от пользователя.
- **Шаблоны**. Это файлы с расширением .html, которые содержат статический контент, такой как текст, изображения и стили. Flask использует шаблоны для создания HTML-страниц, которые будут отправлены пользователю.
- **Контекст**. Это объект, который содержит информацию о текущем запросе и ответе. Контекст используется для передачи данных между различными частями приложения.
- **Расширения**. Это модули, которые добавляют в Flask новые функции.

Существует множество расширений для Flask, таких как поддержка баз данных, аутентификация пользователей, кеширование и так далее по списку.

Вот несколько примеров того, как можно использовать расширения Flask:

- **Поддержка баз данных.** С помощью расширения Flask-SQLAlchemy можно легко подключить к приложению базу данных MySQL или PostgreSQL. Это позволит хранить данные пользователей, товаров, заказов и т. п.
- **Аутентификация пользователей.** Расширение Flask-Login позволяет реализовать аутентификацию пользователей по логину и паролю. Это защитит приложение от несанкционированного доступа.
- **Кеширование.** Расширение Flask-Caching позволяет кэшировать результаты запросов к базе данных или другим внешним источникам. Это ускорит работу приложения и снизит нагрузку на сервер.
- **Email-уведомления.** Расширение Flask-Mail позволяет отправлять электронные письма пользователям. Это может быть полезно для отправки уведомлений о новых сообщениях, заказах и т. п.
- **Работа с файлами.** Расширение Flask-FileUpload позволяет загружать файлы на сервер. Это может пригодиться для загрузки изображений, документов и других файлов.
- **Тестирование.** Расширение Flask-Testing предоставляет инструменты для тестирования приложений Flask. Это поможет убедиться, что приложение работает правильно.
- **Международные символы.** Расширение Flask-Babel позволяет переводить текст на другие языки. Это сделает приложение доступным для пользователей из разных стран.
- **Авторизация.** Расширение Flask-Security позволяет реализовать авторизацию пользователей на основе ролей. Это ограничит доступ пользователей к определённым функциям приложения.
- **Сеансы.** Расширение Flask-Session позволяет сохранять данные о состоянии пользователя между запросами. Это пригодится для сохранения корзины покупок, текущего местоположения пользователя и т. п.

Что такое FastAPI?

FastAPI — это один из немногих современных фреймворков, обладающий высокой производительностью для разработки веб-сервисов, написанный на Python. FastAPI, в отличии от Flask, был создан в 2019г. и имел несколько преимуществ перед Flask. FastAPI основан на принципах быстрой разработки и обеспечивает лёгкость в создании своего собственного сайта. Основное преимущество FastAPI это то, что он использует подход, основанный на типизации данных, что позволяет автоматически генерировать документацию по API и обеспечивает проверку типов входных данных. Это делает разработку более эффективной и безопасной.

Основными особенностями FastAPI являются:

- высокая производительность;
- простота использования и лаконичность кода;
- поддержка различных форматов данных;
- встроенная поддержка валидации данных;
- возможность интеграции с базами данных и другими сервисами.

Так же в FastAPI есть различные расширения для того, чтобы упростить какую ни будь функцию или программу. Есть множество различных инструментов и библиотек для FastAPI. Вот некоторые из них:

- **Pydantic**: библиотека для проверки типов данных, которая используется в FastAPI. Она позволяет определять схемы данных и проверять входные данные на соответствие этим схемам.
- **Swagger UI**: инструмент для создания документации по API. Он интегрируется с FastAPI и позволяет создавать интерактивные документы, которые описывают параметры запросов и ответов.
- **SQLAlchemy**: ORM (Object-Relational Mapping) для работы с базами данных. Он позволяет абстрагироваться от деталей работы с базой данных и сосредоточиться на бизнес-логике приложения.
- **Alembic**: инструмент для управления миграциями баз данных. Он помогает автоматизировать процесс изменения структуры базы данных при изменении модели данных.
- **Gunicorn**: серверный веб-сервер, который может использоваться для развёртывания приложений на основе FastAPI. Он обеспечивает высокую производительность и надёжность работы.
- **Uvicorn**: альтернативный веб-сервер для FastAPI, который также обеспечивает высокую производительность. Он имеет более простой и понятный интерфейс, чем Gunicorn.
- **Redis**: распределённая система хранения данных, которую можно использовать для кэширования результатов запросов к базе данных или других данных. Она обеспечивает быстрый доступ к данным и повышает производительность приложения.
- **Celery**: очередь задач, которая позволяет выполнять фоновые задачи и асинхронные операции. Она может быть полезна для таких задач, как отправка уведомлений, обработка файлов и т. д.

- **JWT (JSON Web Token)**: стандарт для аутентификации и авторизации пользователей. Он позволяет передавать информацию о пользователе в зашифрованном виде и проверять её на сервере.

Как уже было сказано ранее, одной из ключевых особенностей FastAPI является его способность быстро и эффективно обрабатывать запросы. Это получается за счёт использования асинхронного программирования, которое позволяет выполнять несколько задач одновременно, и в результате мы получаем FastAPI который может обрабатывать больше запросов без снижения производительности чем тот же Django.

FastAPI также предоставляет разработчикам множество возможностей для настройки и расширения функциональности. Как было сказано ранее этот фреймворк поддерживает разные типы данных JSON, XML и другие форматы. Кроме этого, FastAPI имеет встроенную систему документации, которая автоматически генерирует эту самую документацию по API на основе кода. Это упрощает процесс разработки и тестирования FastAPI.

Так же ещё одна основная особенностей FastAPI является его простота и понятность. Интерфейс FastAPI прост и интуитивно понятен, что делает его доступным для любых даже начинающих разработчиков. В официальной документации FastAPI есть множество примеров и руководств, которые помогают разработчикам быстро начать работу с этой библиотекой.

В целом, FastAPI представляет собой мощный и гибкий инструмент для разработки веб-приложений на Python. Его высокая производительность, поддержка типов данных, встроенная документация и расширяемость делают его привлекательным выбором для разработчиков, которым требуется создать RESTfull API с использованием принципов быстрой разработки.

Что такое Django?

Django — это высокоуровневый фреймворк на языке Python, который предназначен для разработки веб-приложений. Он помогает разработчикам создавать безопасные и масштабируемые проекты, предоставляя набор готовых функций и инструментов.

Основные особенности Django:

- **Безопасность.** Фреймворк обеспечивает защиту от распространённых уязвимостей, таких как SQL-инъекции, XSS-атаки и CSRF-атаки. Это позволяет разработчикам сосредоточиться на бизнес-логике приложения, не беспокоясь о безопасности.
- **Масштабируемость.** Django поддерживает горизонтальное масштабирование, что позволяет легко увеличивать производительность приложения при увеличении нагрузки.
- **Скорость разработки.** Фреймворк предоставляет множество готовых решений, таких как ORM (Object-Relational Mapping), формы, аутентификация и авторизация, которые позволяют быстро создавать функциональные веб приложения.
- **Сообщество и документация.** Django имеет большое и активное сообщество разработчиков, а также обширную документацию, которая помогает новичкам

быстро начать работу с фреймворком. Так же для Django существует массы различных модификаций и библиотек

Вот некоторые из основных компонентов Django:

- **ORM (Object-Relational Mapping).** Предоставляет абстракцию над базой данных, позволяя разработчикам работать с объектами Python вместо SQL-запросов.
- **Формы.** Позволяют легко создавать и обрабатывать формы ввода данных от пользователей.
- **Аутентификация и авторизация.** Обеспечивают безопасность приложения, позволяя пользователям проходить аутентификацию и получать доступ к определённым функциям в зависимости от их роли.
- **Административный интерфейс.** Предоставляет простой способ управления данными приложения через веб-интерфейс.
- **Шаблоны.** Позволяют отделять логику приложения от представления, обеспечивая гибкость и удобство в разработке.

Django также имеет множество расширений и библиотек, которые можно использовать для создания более сложных проектов. Например, Django REST Framework позволяет создавать REST API, а Django Channels — интегрировать асинхронное программирование для обработки запросов в реальном времени.

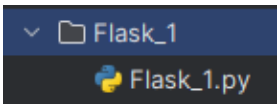
Фреймворк Django может быть использован для различных целей, включая разработку веб-сайтов, порталов, социальных сетей, блогов и других проектов. Он предоставляет разработчикам мощный и гибкий инструмент для создания безопасных и масштабируемых веб-приложений на Python.

Теперь, после ознакомления и представления данных фреймворках, осталось только посмотреть, как каждый из них работает.

Примеры работы программы на Django, FastAPI и Flask

Пример работы на Flask

Для того чтобы начать работу на Flask нужно установить его с помощью консольной команды “**`pip install Flask`**”. После этого достаточно просто создать новый файл с расширением “**`.py`**”.(рис. 1.1)



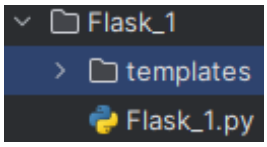
(рис. 1.1)

Потом обязательно нужно импортировать все нужные модули. Во-первых, сам **`Flask`**, а во-вторых, для сайта понадобиться **`render_template`** (рис. 1_2).

```
from flask import Flask
from flask import render_template
```

(рис. 1.2)

Следующем шагом будет создание директории под названием **`templates`**, это будет директория с различными шаблонами. Создавать директорию следует на том же уровне, где и находиться ваш файл с тем самым расширением “**`.py`**” (рис. 1.3).



(рис. 1.3)

Но перед тем, как начать прописывать функции, нужно сначала создать переменную, которая будет называться **`app`**, она будет принимать наш импортируемый класс **`Flask`** (рис 1.4).

```
app = Flask(__name__)
```

(рис 1.4)

Теперь можно приступать к созданию функций в Flask (рис.1.5).

```
@app.route('/hello_1')
def hello_1():
    text = 'Привет 1'
    return render_template( template_name_or_list: 'Hello_1.html', text_1=text)


@app.route('/hello_2')
def hello_2():
    text = 'Привет 2'
    return render_template( template_name_or_list: 'Hello_2.html', text_2=text)
```

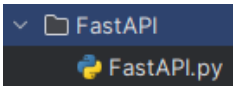
(рис.1.5)

В данном случае “**`.route`**” это наше подключение на страницу, указанную в скобках. В “**`.route`**” указываем путь “**`(/hello_1)`**” и “**`(/hello_2)`**”, если что путь

можно указывать какой душе угодно. Дальше идёт переменная **text**, которая содержит строку, а после функция возвращает **render_template** в котором указаны шаблоны “**Hello_1**” и “**Hello_2**” с расширением “**.html**”.

Пример работы на FastAPI

Перед началом работы с FastAPI нужно его установить многого для этого не надо, так как запуск FastAPI схож с Flask. Сначала также устанавливаем FastAPI “**pip install fastapi**” (рис. 2.1).



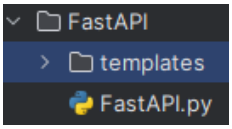
(рис. 2.1)

Теперь также, как и во Flask импортируем из **fastapi** класс **FastAPI**, а по пути **fastapi.templating** импортируем **Jinja2Templates** (рис. 2.2).

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates
```

(рис. 2.2)

После создаём директорию и называем её **templates**, эта директория будет хранить в себе **HTML-шаблоны**, то есть шаблоны с расширением “**.html**” (рис. 2.3).



(рис. 2.3)

Теперь создаём две переменные с названием app и templates. Переменная app будет содержать в себе класс **FastAPI**, а **templates** будет содержать **Jinja2Templates** с указанным путём в папку с шаблонами (рис. 2.4).

```
app = FastAPI()
templates = Jinja2Templates(directory='templates')
```

(рис. 2.4)

А сейчас можно непосредственно переходить к самим функциям (рис. 2.5).

```
@app.get(path='/hello')
def hello_1(request: Request):
    return templates.TemplateResponse('Hello_1.html', {'request': request, 'text': 'Привет'})


@app.get(path='/hello_2')
def hello_2(request: Request):
    text_1 = 'Привет 1'
    text_2 = 'Привет 2'
    content = {'request': request,
               'text_1': text_1,
               'text_2': text_2}
    return templates.TemplateResponse('Hello_2.html', content)
```

(рис. 2.5)

Как уже можно заметить вместо **app.route()** как во Flask стоит **app.get()**. В FastAPI **.get()** является одним из семи запросов. На самом деле в FastAPI кроме **.get()** есть ещё **.put()**, **.post()**, **.patch()**, **.head()**, **.options()** и

.delete() запросы и весь ваш сайт будет работать через них, если вы делайте это в FastAPI.
В FastAPI каждый подобный запрос определяет, что будет с файлом, то есть будет ли этот файл просто вызван или удалён. Ниже приведены все запросы в FastAPI:

- Метод **.get()** это запрос позволяющий получить данные сервера
- Метод **.delete()** всем своим видом показывает, что он удаляет файл или файлы в базе данных.
- Метод **.patch()** подразумевает частичное изменение базы данных.
- Метод **.put()** — это метод для создания или изменения базы данных
- Метод **.head()** сделан для получения статуса запроса или содержимом ответа без погрузки всего объёма.
- Метод **.options()** даёт информацию о доступных методах

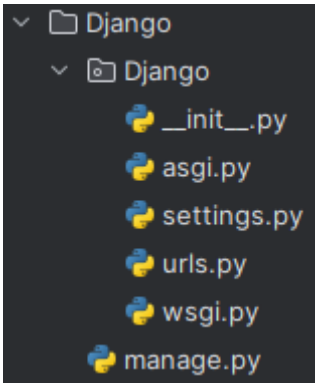
На данный момент это все существующие запросы в FastAPI. Может быть, в будущих обновлениях в этом фреймворке добавят новые запросы кроме представленных, но такое вряд ли произойдёт, как минимум из-за того, что разработчикам пока что вполне достаточно этих запросов

Пример работы на Django

Для начала нужно установить Django, делается это так же, как и в предыдущих примерах просто в консоли нужно прописать "**pip install Django**". В отличии от FastAPI и Flask, в Django чтобы начать разработку проекта, в консоли нужно прописать "**django-admin startproject {имя файла}**" (рис. 3.1) и (рис. 3.2).

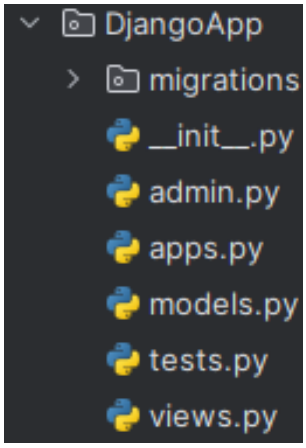
```
> django-admin startproject Django
```

(рис. 3.1)



(рис. 3.2)

Все файлы затрагивать не будем, а пробежимся только по тем которые помогут создать сайт. Но перед тем, как создать его нужно создать приложение в директории **Django**. С помощью команды **cd {имя директории}** переходим в папку **Django** и прописываем команду в консоли "**python manage.py startapp {имя директории}**" (рис. 3.3).



(рис. 3.3.1)

```
> python manage.py startapp DjangoApp
```

(рис. 3.3.2)

В приложении также не будем пробегаться по всем файлам, а пройдёмся только по основному. И в основном нам понадобятся файл **`views.py`** в приложении, и файлы **`urls.py`** и **`settings.py`**, в проекте Django.

Для начала разберёмся с **`settings.py`**, а конкретнее в нём нам надо прописать наше приложение в лист **`INSTALLED_APPS`** (рис3.4).

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'DjangoApp']
```

(рис. 3.4)

Теперь в **`TEMPLATES`**, а конкретнее в листе “**`DIRS`**” нужно прописать вашу папку с шаблонами с помощью **`BASE_DIR`** (рис. 3.5).

```
TEMPLATES = [{
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',], }, },]
```

(рис. 3.5)

Теперь файл **`settings.py`** нам больше не понадобиться. Но стоит учесть, что при прописи вашей папки с шаблонами нужно ставить её на уровень вашего приложения, а также указывать полный путь по директориям если они там есть,

например: “[**BASE_DIR / ‘templates/task’]**”. После настройки проекта можно переходить к функциям.

Все функции создаются в файле **views.py**. В этом файле уже есть импортированный **render** и нам этого более чем достаточно. Теперь просто прописываем функции не забывая возвращать **render**. В нём мы указываем сам **request** который получаем потом указываем шаблон “**Hello_1.html**” и “**Hello_2.html**”. И потом указываем либо переменную, в которой обязательно записана в виде словаря, либо сам словарь (рис. 3.6).

```
from django.shortcuts import render
# Create your views here.


def hello_1(request):
    text_1 = 'Привет 1'
    text_2 = 'Привет 2'
    context = {'text_1': text_1,
               'text_2': text_2}
    return render(request, template_name: 'Hello_1.html', context)


def hello_2(request):
    return render(request, template_name: 'Hello_1.html', context: {'text_3': 'Привет'})
```

(рис. 3.6)

Теперь переходим в папку **urls.py** в ней нам нужно прописать все пути, которые мы хотим сделать с помощью **path**. В кавычках прописываем сам путь, а через запятую функцию. Саму функцию следует импортировать из **views.py** наши функции (рис. 3.7).

```
from django.contrib import admin
from django.urls import path
from DjangoApp.views import *

urlpatterns = [
    path('admin/', admin.site.urls),
    path('hello_1', hello_1),
    path('hello_2', hello_2),]
```

(рис. 3.7)

И наконец, чтобы запустить проект осталось прописать в консоли команду “**python manage.py runserver**” (рис. 3.9), но для того, чтобы питон заметил **manage.py** нужно сначала перейти туда с помощью команды “**cd {имя директории}**” (рис. 3.8)

```
> cd Django
```

(рис. 3.8)

```
> python manage.py runserver
```

(рис. 3.9)

Шаблоны HTML

В первом шаблоне “**Hello_1.html**” кроме всего основного передаётся один аргумент этим аргументом является text который содержит в себе строку “**Привет 1**”.

Во втором шаблоне под названием “**Hello_2.html**” передаются переменные **text_1** и **text_2**. Переменная **text_1** содержит в себе строку “**Привет 2**”, а в **text_2** строка “**Привет 3**”. Ну а выглядит всё это так (рис. 4.1).

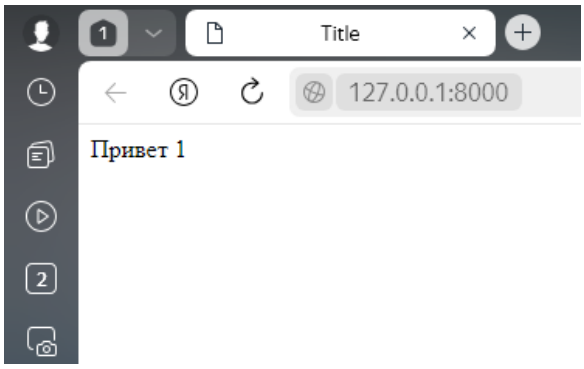
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    {{ text }}
</body>
</html>
```

(рис. 4.1.1)

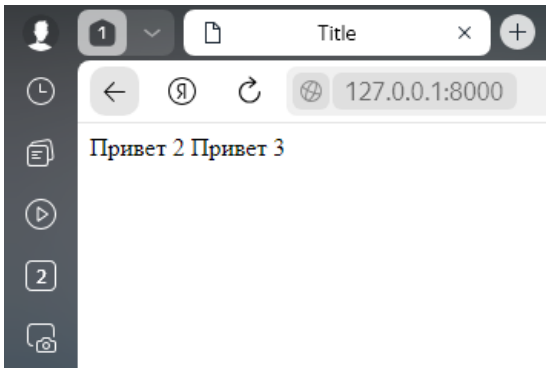
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    {{ text_1 }}
    {{ text_2 }}
</body>
</html>
```

(рис. 4.1.2)

Ни один из фреймворков кроме Django не ограничивает вас в использовании или изменении шаблонов HTML. Каждый созданный вами шаблон можно использовать полностью без ограничений, но стоит учесть, что Django ограничивает логическую обработку в HTML-шаблонах. Ниже вы можете увидеть, как выглядит сам сайт, в каждом фреймворке одинаково (рис. 4.2).



(рис. 4.2.1)



(рис.4.2.2)

Сравнение работы на Flask, Django и FastAPI

В этой главе я всё буду сравнивать в заготовленной последовательности. Сначала я сравню Django и FastAPI, затем Django с Flask, потом я сравню FastAPI с Flask, и в конце я выделю все плюсы и минусы по итогу этих сравнений.

Сравнение Django и Flask

Сейчас будет сравнение Django и FastAPI, а после сравнения будет проведён анализ и выделены основные плюсы этих двух фреймворков.
Плюсы Django:

1. **Стандартизированная структура**
   Django как фреймворк задаёт структуру проекта. Она помогает разработчикам понимать, где и как добавлять новую функциональность.
   Благодаря одинаковой для всех проектов структуре гораздо проще найти уже готовые решения или получить помощь от сообщества. Огромное количество увлеченных разработчиков поможет справиться с любой задачей гораздо быстрее.
2. **Безопасность.**
   Django имеет встроенные функции безопасности, которые помогают предотвратить распространённые уязвимости, такие как SQL-инъекции, XSS-атаки и CSRF-атаки. Это делает его идеальным выбором для разработки безопасных веб-приложений.
3. **Многообразие дополнительных модификаций и библиотек.**
   Django за всё своё время существования сформировал большое и активное сообщество разнообразных разработчиков, которые создают множество различных модификаций, а это в свою очередь даёт большой простор для разработки
4. **Масштабируемость.**
   Django может использоваться для создания как небольших, так и крупных веб-приложений. То есть мы можем использовать Django как для какого ни будь небольшого сайта, так и для сайта, которым в будущем может владеть какая ни будь сверх корпорация.
5. **ORM (Object-Relational Mapping).**
   Django использует ORM для работы с базами данных. Это упрощает процесс взаимодействия с базой данных и позволяет вам сосредоточиться на логике вашего приложения. Сам ORM (Object-Relational Mapping) предоставляет абстракцию над базой данных, позволяя разработчикам работать с объектами Python вместо SQL-запросов.
6. **Шаблоны и представления.**
   Django предлагает мощные механизмы для работы с шаблонами и представлениями, что позволяет вам создавать динамические и интерактивные веб-страницы.
7. **Управление статическими файлами.**
   Django упрощает управление статическими файлами, такими как CSS, JavaScript

и изображения, что облегчает процесс развёртывания и обновления вашего приложения.

8. **Поддержка MVC (Model-View-Controller) архитектуры.**
   Django поддерживает архитектуру MVC, которая разделяет ваше приложение на три компонента: модель, представление и контроллер. Это улучшает организацию кода и упрощает разработку.

Плюсы Flask:

1. **Лёгкость и гибкость.** Flask имеет небольшой размер и простую структуру, что делает его лёгким в изучении и использовании. Он предоставляет базовые функции для разработки веб-приложений, но не ограничивает вас в выборе дополнительных инструментов и библиотек.
2. **Модульность:** веб-фреймворк подразумевает модульное программирование, что помогает избегать путаниц и сохранить порядок «на рабочем месте». При этом фреймворк позволяет разработчику брать контроль над структурой проекта и самостоятельно выбирать инструменты.
3. **Расширяемость и комьюнити:** микрофреймворк весьма дружелюбен к новым плагинам. Он просто интегрируется с другими библиотеками и расширениями. Самое приятное — это то что подобные модификации можно легко найти в открытом доступе, так как у Flask, есть большое и активное комьюнити, поэтому вы без труда найдете ответы на все, или почти все свои вопросы и нужные модификации, библиотеки и расширения.

Исходя из плюсов этих двух фреймворков, можно понять, что Django куда более сложнее в программировании из чего следует понимать, что для новичков он может оказаться просто не по зубам, а вот Flask наоборот более приветлив к новичкам, в отличии от Django, но во Flask нельзя создать какой нибудь масштабный проект, и к тому же работать в Flask не очень безопасно из-за RESTfull API.

Сравнение Django и FastAPI

Плюсы Django:

1. **Стандартизированная структура**
   Django как фреймворк задаёт структуру проекта. Она помогает разработчикам понимать, где и как добавлять новую функциональность.
   Благодаря одинаковой для всех проектов структуре гораздо проще найти уже готовые решения или получить помощь от сообщества. Огромное количество увлеченных разработчиков поможет справиться с любой задачей гораздо быстрее.
2. **Безопасность.**
   Django имеет встроенные функции безопасности, которые помогают предотвратить распространённые уязвимости, такие как SQL-инъекции, XSS-атаки и CSRF-атаки. Это делает его идеальным выбором для разработки безопасных веб-приложений.
3. **Многообразие дополнительных модификаций и библиотек.**
   Django за всё своё время существования сформировал большое и активное

сообщество разнообразных разработчиков, которые создают множество различных модификаций, а это в свою очередь даёт большой простор для разработки

4. **Масштабируемость.**
   Django может использоваться для создания как небольших, так и крупных веб-приложений. То есть мы можем использовать Django как для какого ни будь небольшого сайта, так и для сайта, которым в будущем может владеть какая ни будь сверх корпорация.
5. **ORM (Object-Relational Mapping).**
   Django использует ORM для работы с базами данных. Это упрощает процесс взаимодействия с базой данных и позволяет вам сосредоточиться на логике вашего приложения. Сам ORM (Object-Relational Mapping) предоставляет абстракцию над базой данных, позволяя разработчикам работать с объектами Python вместо SQL-запросов.
6. **Шаблоны и представления.**
   Django предлагает мощные механизмы для работы с шаблонами и представлениями, что позволяет вам создавать динамические и интерактивные веб-страницы.
7. **Управление статическими файлами.**
   Django упрощает управление статическими файлами, такими как CSS, JavaScript и изображения, что облегчает процесс развёртывания и обновления вашего приложения.
8. **Поддержка MVC (Model-View-Controller) архитектуры.**
   Django поддерживает архитектуру MVC, которая разделяет ваше приложение на три компонента: модель, представление и контроллер. Это улучшает организацию кода и упрощает разработку.

Плюсы FastAPI:

1. **Скорость работы.** FastAPI обходит все Python-фреймворки по производительности.
2. **Гибкость на уровне Flask.** У вас нет какой-либо утвержденной архитектуры, что дает волю вашей фантазии и различным подходам разработки, по сравнению с Django (использует MVC), в котором проект часто превращается в странную субстанцию из папок в папках.
3. **Автоматическая OpenAI документация.** В Django и Flask, нужно подключать отдельные пакеты и создавать специальные классы для подключения документации к ним. Еще замечу, что к Flask очень мало информации на эту тему, а сама версия документации зачастую устаревшая. На Django это сделать уже намного проще, но также занимает некоторое время, в то же время в FastAPI все доступно из коробки.
4. **Асинхронность.** FastAPI использует ASGI-сервера по умолчанию, когда в том же Django нужно заниматься конфигурацией приложения с WSGI на ASGI, что занимает достаточно времени. Flask, к сожалению, не поддерживает асинхронность и работает лишь под WSGI.
5. **Встроенная валидация данных.** Недавно вышла поддержка Pydantic V2 в FastAPI, что ускоряет валидацию данных в 20 раз. Pydantic предоставляет

готовые типы данных для валидации email, uuid, url и т.д. Вместо того, чтобы вручную писать проверки для содержимого json-файлов и обрабатывать ошибки валидации, он автоматизирует эти процессы.

6. **Проработанная документация.** В случае других фреймворков и библиотек приходится смотреть обучающие ролики из-за невнятной и громоздкой документации. У FastAPI таких проблем нет. Документация разделена на две части: для новичков и более продвинутых разработчиков. В ней большое количество примеров и лучших практик, что также является несомненным плюсом.

Подводя итог по сравнению FastAPI и Django, можно с уверенностью сказать, что FastAPI это самый быстрый фреймворк из всех трёх, но также он является довольно молодым, что означает малое количество модификаций и библиотек, для него. Так же FastAPI благодаря своей документации является очень дружелюбным к новичкам. Но из-за не надёжной безопасности RESTfull API стоит всегда быть на стороже

Сравнение FastAPI и Flask

Плюсы FastAPI:

1. **Скорость работы.** FastAPI обходит все Python-фреймворки по производительности.
2. **Гибкость на уровне Flask.** У вас нет какой-либо утвержденной архитектуры, что дает волю вашей фантазии и различным подходам разработки, по сравнению с Django, в котором проект часто превращается в странную субстанцию из папок в папках.
3. **Автоматическая OpenAI документация.** В Django и Flask, нужно подключать отдельные пакеты и создавать специальные классы для подключения документации к ним. Еще замечу, что к Flask очень мало информации на эту тему, а сама версия документации зачастую устаревшая. На Django это сделать уже намного проще, но также занимает некоторое время, в то же время в FastAPI все доступно из коробки.
4. **Асинхронность.** FastAPI использует ASGI-сервера по умолчанию, когда в том же Django нужно заниматься конфигурацией приложения с WSGI на ASGI, что занимает достаточно времени. Flask, к сожалению, не поддерживает асинхронность и работает лишь под WSGI.
5. **Встроенная валидация данных.** Недавно вышла поддержка Pydantic V2 в FastAPI, что ускоряет валидацию данных в 20 раз. Pydantic предоставляет готовые типы данных для валидации email, uuid, url и т.д. Вместо того, чтобы вручную писать проверки для содержимого json-файлов и обрабатывать ошибки валидации, он автоматизирует эти процессы.
6. **Проработанная документация.** В случае других фреймворков и библиотек приходится смотреть обучающие ролики из-за невнятной и громоздкой документации. У FastAPI таких проблем нет. Документация разделена на две части: для новичков и более продвинутых разработчиков. В ней большое количество примеров и лучших практик, что также является несомненным плюсом.

Плюсы Flask:

4. **Лёгкость и гибкость.** Flask имеет небольшой размер и простую структуру, что делает его лёгким в изучении и использовании. Он предоставляет базовые функции для разработки веб-приложений, но не ограничивает вас в выборе дополнительных инструментов и библиотек.
5. **Гибкость и модульность:** веб-фреймворк предусматривает модульное программирование, что помогает избежать путаницы и сохранить порядок «на рабочем месте». При этом фреймворк позволяет разработчику брать контроль над структурой проекта и самостоятельно выбирать инструменты.
6. **Расширяемость и комьюнити:** микрофреймворк очень дружелюбен к дополнительным плагинам. Он просто интегрируется со сторонними библиотеками и расширениями. Самое приятное — подобные плагины легко найти в открытом доступе. У Flask, как я уже написал, есть большое и активное комьюнити, поэтому вы без труда найдете ответы на свои вопросы и нужные расширения.

В итоге получаем что FastAPI и Flask очень схожи во многих своих аспектах, но всё же это разные фреймворки, как я уже говорил FastAPI относительно молодой фреймворк и из этого следует то, что у него малое количество модификаций и библиотек. Так же оба эти фреймворка не слишком защищенны от угроз из-за RESTfull API.

Подводя предпоследние итоги по сравнению Django, Flask и FastAPI, можно с уверенностью ответить на все заданные мною вопросы по самому подходящему фреймворку.

Для начала выделим все плюсы и минусы в фреймворках Django, FastAPI и Flask:

1. Django:
   - Фреймворк, который помогает в создании любых сайтов и проектов и не важно будут они маленькие или большие
   - Django не подходит для начинающих программистов из-за своих поистине гигантских объёмов и сложности в целом.
2. FastAPI:
   - Можно создавать сайты или проекты, которые будут ограничены в своих возможностях и перспективах. Эти проекты называют RESTfull API
   - В этом фреймворке, есть асинхронное программирование, что даёт FastAPI лучшие параметры производительности и оптимизации среди, если не всех, то подавляющего большинства фреймворков.
   - Имея понятную документацию FastAPI, является хорошим фреймворком, как для начинающих, так и для более прожжённых программистов.
   - Не слишком безопасный фреймворк, стоит быть внимательнее.
3. Flask:
   - Так же, как и FastAPI, является хорошим фреймворком для тех, кто только начал свой путь программиста.
   - И если опять вспоминать как там у FastAPI то здесь так же можно создавать сайты или проекты, которые будут ограничены в своих возможностях и перспективах. Эти проекты называют RESTfull API

- Не слишком безопасный фреймворк, стоит быть внимательнее.

Подводя теперь уже последние итоги, можно с уверенностью разделить все три фреймворка на определённых программистов.

1. Django лучше всего подходит для тех, кто хочет создать большой проект, и кто уже давно занимается программированием и разбирается в некоторых консольных командах и библиотеках.
2. FastAPI подходит для всех начинающих и любых других программистов благодаря своей документации. FastAPI в отличии от Django и Flask является самым производительным и оптимизированным фреймворком, но самая большая проблема заключается в том, что в FastAPI трудно создавать проекты на уровне Django. Максимум который позволяет FastAPI это RESTfull API.
3. Flask будет отличным фреймворком для тех же новичков, но новичкам всё же стоит быть внимательнее так как Flask всё-таки немного сложнее FastAPI, а эти сложности могут вызвать у новичков или любителей затруднения.

Приложение 1. Пример файловой структуры проекта

```
Diplom
| Django
| | Django
| | | __init__.py
| | | asgi.py
| | | settings.py
| | | urls.py
| | | wsgi.py
| | DjangoApp
| | | migrations
| | | | __init__.py
| | |  __init__.py
| | | admin.py
| | | apps.py
| | | models.py
| | | tests.py
| | | views.py
| | templates
| | | Hello_1.html
| | | Hello_2.html
| | __init__.py
| | db.sqlite3
| | manage.py
| FastAPI
| | templates
| | | Hello_1.html
| | | Hello_2.html
| | FastAPI.py
| Flask
| | templates
| | | Hello_1.html
| | | Hello_2.html
| | Flask.py
```

Приложение 2. Список необходимых библиотек

```
Django==4.2.16
fastapi==0.115.0
uvicorn==0.32.0
Flask==3.0.3
```